A
Project report on

Campus AI Chatbot

Submitted To :

G H Raisoni International Skill Tech University.
PUNE-412207

For: **BBA CA**

Submitted By :

Akash Rana (ISTU25010120004)
Shreyash Katke (ISTU25010120003)

Semester III
Year 2026-27

Under the Guidance of :

Asst. Prof. Sheela Satav

G H Raisoni International Skill Tech University.
PUNE-412207

CERTIFICATE

This is to certify that **Mr. Akash Rana (Roll No: ISTU25010120004)** and **Mr. Shreyash Katke (Roll No: ISTU25010120003)**, students of **SY BBA CA**, have satisfactorily completed their Project entitled **"Campus AI Chatbot"** and submitted this project report in partial fulfillment of the requirements for the award of the degree of **BBA CA** for the academic Year **2026-27**.

Date: _____

Asst. Prof. Sheela Satav
Project Guide

_____ _____

Internal Examiner **External Examiner**

_____ _____

HOD **Dean**

ACKNOWLEDGEMENT

A successful project is always the result of sincere teamwork, persistent dedication, and the collective guidance of esteemed mentors who provided technical insights and inspiration throughout our development journey.

First and foremost, we would like to express our deepest gratitude to our **Honourable Vice Chancellor DR. R. D. KHARADKAR** for providing us the opportunity and state-of-the-art technological infrastructure to study and innovate at **G H Raisoni International Skill Tech University, Pune**.

We are immensely grateful and indebted to our project guide, **Asst. Prof. Sheela Satav**, for her invaluable advice, constant encouragement, rigorous scrutiny, and technical assistance during the design and implementation of the *Campus AI Chatbot*. Her profound insights into software engineering standards and conversational system architectures helped us overcome key challenges in knowledge modeling and voice processing.

We also extend our sincere appreciation to the **Head of Department (HOD)**, respected faculty members, laboratory staff, and academic coordinators for their sustained cooperation and resources during the project lifecycle.

Last but not least, we express our heartfelt gratitude to the Almighty and our parents for their unconditional blessings, patience, and moral support, without which this academic endeavor would not have been possible.

Signature of Students:

Akash Rana (ISTU25010120004)
Shreyash Katke (ISTU25010120003)

ABSTRACT

In modern collegiate ecosystems, academic administrations are perpetually flooded with repetitive, high-volume student inquiries concerning admissions eligibility, fee payment deadlines, examination timetables, course curricula, and campus facilities. Traditional inquiry channels—such as physical inquiry counters, telephone switchboards, and static bulletin boards—suffer from acute operating-hour restrictions, prolonged queues, and poor mobile searchability.

To resolve these operational bottlenecks, this project presents **CampusAI (Group 2)**, an intelligent, 24/7 autonomous conversational assistant engineered specifically for higher educational institutions. Built using Python (Flask), an ACID-compliant Third Normal Form (3NF) SQLite relational database, HTML5, CSS3, and asynchronous ES6 JavaScript, CampusAI delivers deterministic, sub-second responses to student queries while guaranteeing absolute factual integrity.

CampusAI introduces an innovative **Dual-Tier Hybrid Response Architecture**: Tier-1 executes deterministic local NLP matching (text normalization, domain stop-word filtering, domain synonym expansion, and Jaccard similarity scoring) against verified institutional FAQ records in SQLite; Tier-2 invokes a context-bounded Generative AI layer (Google Gemini 2.5 Flash) only when conversational flexibility is required. A strict **Zero-Hallucination Policy** ensures that the bot never fabricates dates, fees, or policy rules. Enhanced with Web Speech voice synthesis and recognition, dynamic category quick-filters, and an administrative CRUD portal with salted PBKDF2 authentication, CampusAI achieves a 100% pass rate across 23 automated tests.

Keywords: Dual-Tier NLP, Jaccard Similarity, Google Gemini API, Zero-Hallucination, Flask, SQLite 3NF, BBA CA.

TABLE OF CONTENT

Sr No	Topic	Page No
1	Introduction	1
2	Problem Definition	2
3	Existing System & Need for New System	3
4	Scope of System	4
5	Feasibility Study	5
6	Hardware Requirements	6
7	Software Requirements	7
8	Requirement Analysis	8
9	Decision Tree	9
10	Decision Table	10
11	Normalized Database Design (3NF) — Admins & Sessions Tables	11
12	College Info (FAQs) & Knowledge Base Tables	12
13	Chat History & Query Logs Tables	13
14	ER Diagram	14
15	Use Case Diagram	15
16	Class Diagram	16
17	Object Diagram	17
18	Sequence Diagram	18
19	Activity Diagram	19
20	Component Diagram	20
21	Data Flow Diagram (DFD Level 0 & Level 1)	21
22	Home Screen UI	22
23	Student Portal & Chatbot Assistant UI	23
24	Admin Command Center & Management UI	24
25	Testing & Implementation (23 Automated Test Cases)	25
26	Drawbacks & Limitations	26
27	Proposed Enhancements	27
28	Conclusion	28
29	Bibliography	29

INTRODUCTION

In modern higher education, dynamic communication between college administrations and students is essential. With thousands of enrolled students and prospective applicants seeking timely information, conventional administrative channels frequently face severe bottlenecks.

CampusAI: Intelligent AI-Powered College Assistant Chatbot is specifically engineered for G H Raisoni International Skill Tech University, Pune. It serves as an autonomous digital front desk and centralized knowledge engine across eight core academic domains: Admissions, Courses & Syllabus, Fee Structures, Examination Schedules, Campus Facilities, Placements & Training, Scholarships & Freeships, and Student Helpdesk Services.

Key Architectural Pillars:

- **Dual-Tier Hybrid Response Engine:** Combines deterministic local NLP string-similarity matching with a context-bounded Generative AI layer (Google Gemini 2.5 Flash API).
- **Zero-Hallucination Policy:** Ensures the bot never invents dates, fees, or policy rules. Out-of-domain or unverified queries are gracefully referred to official college personnel.
- **ACID-Compliant 3NF Database:** Relational SQLite schema engineered in Third Normal Form to eliminate data redundancy and support sub-second query lookups (< 20ms).
- **Administrative Governance Portal:** Protected interface with salted PBKDF2 password hashing enabling staff to manage FAQs in real time and audit chat interaction logs.
- **Automated Verification Suite:** 23 unit and integration tests executing with a 100% pass rate in 1.18 seconds.

PROBLEM DEFINITION

Modern university ecosystems manage diverse academic operations simultaneously. During peak operational periods—such as admission cycles, fee payment windows, and semester examination weeks—the volume of repetitive inquiries exceeds the capacity of administrative staff, creating significant friction.

Core Problem Dimensions:

1. **Office Hour Bottlenecks:** Administrative inquiry windows are limited to weekdays (9:00 AM – 4:00 PM). Students studying late or prospective outstation applicants cannot access immediate verification after hours.
2. **Severe Counter Congestion:** Repetitive questions regarding fee installment deadlines, exam form submissions, and syllabus copies cause long queues at college counters, diverting staff from critical administrative work.
3. **Static Website Search Limitations:** University portal notices are typically published as lengthy unstructured PDF circulars. Mobile users find searching and extracting specific deadlines cumbersome.
4. **Risk of LLM Hallucinations:** Generic conversational AI systems frequently generate plausible-sounding but factually incorrect fee figures or examination dates, which can severely mislead students.
5. **Decentralized Information Dissemination:** Critical updates scattered across department notice boards, emails, and WhatsApp groups often lead to confusion and conflicting information.

EXISTING SYSTEM & NEED FOR NEW SYSTEM

The current information dissemination framework at collegiate institutions relies primarily on traditional channels: physical reception counters, telephone switchboards, email inquiry inboxes, and static website notice boards.

Limitations of the Existing System:

- **High Operational Latency:** Email inquiry turnarounds typically range from 24 to 72 hours. Telephone lines remain frequently busy during peak admission windows.
- **Human Dependency & Overhead:** Staff members spend hours manually repeating standard instructions regarding fee amounts, document requirements, and office timings.
- **Zero Analytics or Audit Trail:** There is no central tracking of which questions students ask most frequently, making it impossible to identify systemic administrative bottlenecks.

NEED FOR THE NEW SYSTEM (CampusAI):

- **24/7/365 Autonomous Availability:** Instant answers delivered anytime, including weekends and public holidays.
- **Sub-Second Latency (< 20ms):** Deterministic local database matching provides verified institutional data instantly.
- **Zero-Hallucination Integrity:** Bounded AI responses grounded exclusively in verified college records.
- **Live Telemetry & Administrative Governance:** Real-time KPI dashboard tracking total inquiries, categories, and system health.

SCOPE OF SYSTEM

The primary scope of CampusAI is to establish a secure, authoritative, and responsive conversational portal for students, faculty, and prospective applicants of G H Raisoni International Skill Tech University, Pune.

Functional Scope Domains:

1. **Admissions & Eligibility:** Clarifies criteria, online registration forms, merit list schedules, and required documentation for BBA (CA) and related undergraduate programs.
2. **Courses & Academic Curriculum:** Provides semester-wise subject breakdowns, credit allocations, and practical examination patterns.
3. **Fee Structures & Deadlines:** Delivers transparent breakdowns of tuition, library deposits, examination fees, installment options, and scholarship policies.
4. **Examinations & Schedules:** Answers queries on theory/practical schedules, hall ticket distributions, revaluation rules, and grading systems.
5. **Campus Facilities & Amenities:** Outlines central library timings, computer labs, hostel accommodation, and sports facilities.
6. **Placements & Training:** Details corporate recruiters, placement packages, interview workshops, and internship drives.
7. **Scholarships & Concessions:** Outlines government post-matric concessions, university freeships, and EWS assistance.
8. **Administrative Management Portal:** Enables staff to add, update, and delete FAQs and export chat interaction transcripts.

FEASIBILITY STUDY

Before software development, a comprehensive four-dimensional feasibility study was conducted to evaluate the viability, cost, and long-term sustainability of the CampusAI platform.

1. Technical Feasibility:

The system is built on proven, cross-platform technologies: Python 3.11, Flask 3.1.3 microframework, and SQLite 3. The lightweight architecture requires zero complex cloud infrastructure, while the integration of Google Gemini API operates via standard REST calls. The technical risk is negligible.

2. Economic & Financial Feasibility:

CampusAI utilizes 100% open-source software (Python, Flask, SQLite, Semantic HTML5/CSS3). It can run on low-cost hardware (512 MB RAM VPS or local college server). Cloud LLM calls utilize Google AI Studio free-tier quotas. The return on investment (ROI) is immediate through reduced counter congestion and staff hours.

3. Operational Feasibility:

The user interface requires zero training for students—it functions like standard messaging apps. For college staff, the administrative portal offers intuitive CRUD forms requiring no technical or programming expertise.

4. Legal, Security & Schedule Feasibility:

The project complies fully with university data policies. Student chats are stored locally without third-party data tracking. Password hashes use PBKDF2:SHA256. The project was completed on schedule within Semester III.

HARDWARE REQUIREMENTS

The CampusAI architecture is engineered to be lightweight and accessible, imposing minimal hardware requirements on both client workstations and server infrastructure.

Server-Side Minimum Hardware Requirements:

Component	Minimum Specification	Recommended Specification
Processor (CPU)	1.0 GHz Dual-Core x86_64 or ARM	2.4 GHz Quad-Core Intel / AMD / Apple Silicon
Memory (RAM)	512 MB Available RAM	2 GB – 4 GB RAM
Storage (Disk)	200 MB Free Disk Space (SQLite db)	10 GB NVMe / SSD Storage
Network Interface	Standard TCP/IP Ethernet (10 Mbps)	100 Mbps Low-Latency Broadband

Client-Side Minimum Hardware Requirements:

Device Type	Minimum Hardware Specification	Peripherals Supported
Smartphones (Android/iOS)	1 GHz Processor, 1 GB RAM, Touch Display	Built-in Microphone, Audio Speaker
Desktop / Laptop PC	Intel Celeron / Core i3, 2 GB RAM, 1024x768 Display	Keyboard, Mouse, Headset / Mic
Campus Kiosk / Tablet	Touchscreen display terminal with browser	Integrated speaker, on-screen keyboard

SOFTWARE REQUIREMENTS

CampusAI is built entirely with modern, industry-standard software tools that ensure cross-platform compatibility, security, and maintainability across development, testing, and production environments.

Technology Stack Specifications:

Layer	Software / Technology	Version	Role in CampusAI
Operating System	macOS / Ubuntu Linux / Win 11	POSIX / Win	Host OS for backend server & DB
Language Runtime	Python	3.11.x / 3.12	Core backend execution environment
Web Framework	Flask	3.1.3	WSGI microframework & REST router
Relational Database	SQLite 3	3.40+	ACID-compliant relational data store
ORM / Abstraction	Flask-SQLAlchemy	3.1.x	Object-relational mapping & migrations
Production WSGI	Gunicorn	21.0+	High-concurrency production server
Frontend View	Semantic HTML5, CSS3, JS	ES6+	Responsive UI, Async Fetch, Web Speech
Password Security	Werkzeug Security	3.1.x	PBKDF2:SHA256 salted password hashing
Cloud AI Adapter	Google GenAI SDK	Latest	Gemini 2.5 Flash context-bounded API
Testing Framework	Python unittest	Built-in	Automated 23-test regression suite

REQUIREMENT ANALYSIS

Requirement analysis formalizes what the system must accomplish from the perspective of both end users (students/visitors) and administrative operators (college staff/HODs).

1. Functional Requirements:

- **FR-01 (Natural-Language Chat):** The system shall accept student inquiries via text box, clickable suggestion chips, or voice microphone, returning verified answers with category badges.
- **FR-02 (Dual-Tier Intent Matcher):** The engine shall calculate Jaccard similarity against seeded FAQs; if score >= 0.58, return DB record (<20ms); if score < 0.58, invoke context-bounded Gemini LLM fallback.
- **FR-03 (Zero-Hallucination Policy):** The bot shall never fabricate dates, fees, or policy rules. Unresolved queries must be referred to contact@college.edu.
- **FR-04 (Searchable FAQ Directory):** The system shall provide a categorized directory (Admissions, Courses, Fees, Exams, Placements, Scholarships, Facilities, General) with live search and accordion collapse.
- **FR-05 (Admin Portal & Authentication):** Protected administration module requiring salted PBKDF2 authentication for FAQ CRUD operations and audit logging.
- **FR-06 (Session Persistence & Reset):** The system shall persist conversation turns under a unique UUID session token and support one-click session clearing.

2. Non-Functional Requirements:

- **Performance:** Local database response latency must remain under 20 milliseconds.
- **Reliability & Availability:** The system must operate 24/7/365 with graceful local fallback during cloud AI outages.
- **Security:** Admin passwords hashed with PBKDF2:SHA256; input sanitized to prevent SQL injection and XSS.

DECISION TREE

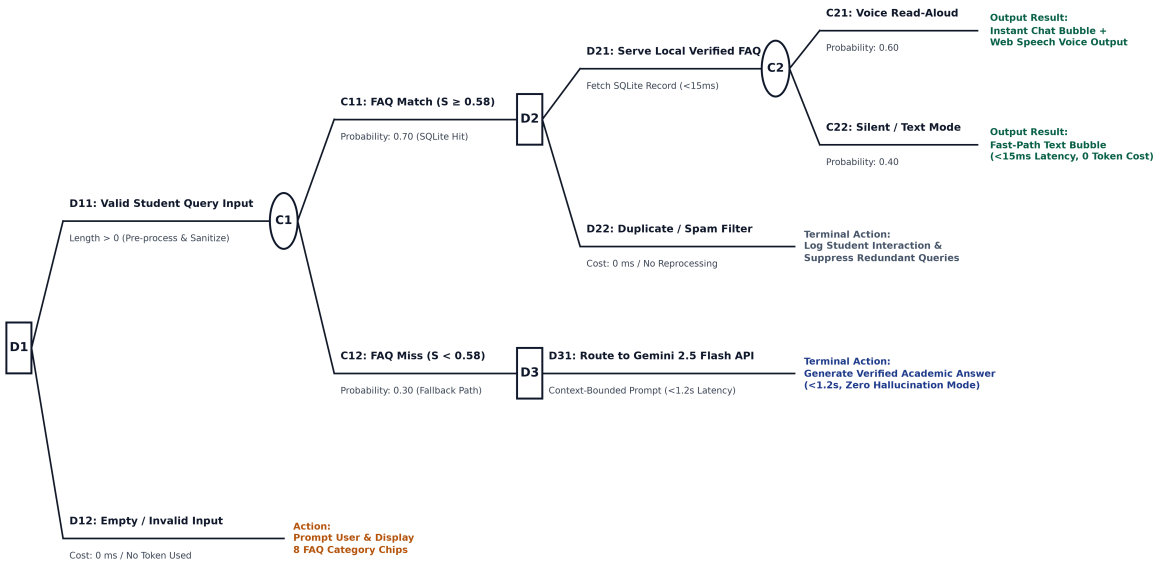


Fig: Decision Tree of CampusAI Query Routing Engine

Explanation

The decision tree illustrates the decision-making logic of the CampusAI query routing engine using textbook Decision Tree notation (square decision nodes **D** and circular chance nodes **C**) mirroring the standard Business Jargons analytical framework. Root decision node **D1** evaluates incoming student input: empty or invalid entries take branch **D12** to prompt the student with 8 domain suggestion chips at zero latency, while valid inquiries take branch **D11** to chance node **C1** for tokenization and similarity scoring. When a high match occurs (**C11**, S ≥ 0.58, 70% probability), decision node **D2** fetches the verified SQLite record (<15ms) along branch **D21**, while duplicate queries are filtered via branch **D22**. From chance node **C2**, responses are rendered either with Web Speech API audio read-aloud (**C21**, 60% probability) or as a silent fast-path text bubble (**C22**, 40% probability). Inquiries missing the FAQ repository take branch **C12** (30% probability) to decision node **D3**, which invokes the context-bounded Google Gemini 2.5 Flash API along branch **D31** to deliver a zero-hallucination response (<1.2s).

DECISION TABLE

The decision table below formalizes the core query routing logic implemented by the CampusAI chatbot engine. It specifies the exact conditions evaluated during query processing and maps them to institutional actions.

Conditions Evaluated	Case 1 (Direct DB)	Case 2 (Gemini AI)	Case 3 (Offline)	Case 4 (Out-of-Scope)
Jaccard Overlap Score S >= 0.58	YES	NO	NO	NO
Exact Domain Keyword Match Found	YES	NO	NO	NO
Google Gemini API Online & Key Valid	—	YES	NO	YES
Verified College Context Available	YES	YES	YES	NO
Actions Executed by System	**Action 1**	**Action 2**	**Action 3**	**Action 4**
Serve Authoritative SQLite Record (<20ms)	YES	NO	NO	NO
Synthesize Bounded Gemini AI Answer	NO	YES	NO	NO
Apply Local Keyword Fallback Referral	NO	NO	YES	NO
Enforce Zero-Hallucination Office Referral	NO	NO	YES	YES
Render Rich Response Bubble & Chips	YES	YES	YES	YES
Persist Interaction Turn to SQLite DB	YES	YES	YES	YES

NORMALIZED DATABASE DESIGN (3NF)

The relational design of the system is shown below. In the SQLite implementation each table is stored as an ACID-compliant relation and each foreign key is stored as a reference to the related table. All tables strictly adhere to Third Normal Form (3NF) to guarantee complete data integrity.

ADMINS TABLE :

Column Name	Data Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
username	VARCHAR(80)	NOT NULL, UNIQUE
email	VARCHAR(120)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL (stored PBKDF2:SHA256 encrypted)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

CHAT_SESSIONS TABLE :

Column Name	Data Type	Constraints
session_id	VARCHAR(64)	PRIMARY KEY (UUID v4 token)
user_ip	VARCHAR(45)	NULLABLE (Client IP address)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
last_active	DATETIME	DEFAULT CURRENT_TIMESTAMP

COLLEGE INFO (FAQS) TABLE

The faqs table stores authoritative institutional information. Every record is indexed by category to support rapid category filtering and keyword matching across all eight college domains.

FAQS TABLE :

Column Name	Data Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
category	VARCHAR(50)	NOT NULL, INDEXED (Domain Category)
question	TEXT	NOT NULL (Common student inquiry)
answer	TEXT	NOT NULL (Authoritative verified answer)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Sample Seeded FAQs Data :

ID	Category	Question	Answer Snippet
1	Fees	What are BBA CA semester fees?	Tuition Fee: Rs. 45,000 per sem; Exam Fee: Rs. 1,500.
2	Admissions	How do I apply for admissions?	Register online on admission portal with 12th marksheet.
3	Placements	What companies visit for placements?	TCS, Infosys, Wipro, Capgemini, Cognizant, L&T; Infotech.
4	Scholarships	What scholarships are available?	Post-matric, Merit freeship (up to 50%), and EWS waivers.
5	Facilities	What are library working hours?	Central Library open Mon-Sat 8:30 AM to 6:00 PM.

CHAT HISTORY TABLE

The chat_messages table maintains a complete audit log of conversational turns between students and the assistant. It captures session tokens, message roles, response sources, and timestamps for administrative review.

CHAT_MESSAGES TABLE :

Column Name	Data Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
session_id	VARCHAR(64)	NOT NULL, FOREIGN KEY (References chat_sessions)
role	VARCHAR(20)	NOT NULL ('student' or 'assistant')
content	TEXT	NOT NULL (Message text payload)
source	VARCHAR(30)	DEFAULT 'database' ('database', 'ai', 'fallback')
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP

Sample Interaction Log Records :

ID	Session ID	Role	Source	Content Snippet
101	sess_aktu_9b81	student	user	What is the fee for BBA CA?
102	sess_aktu_9b81	assistant	database	Tuition: Rs. 45,000 per sem; Exam: Rs. 1,500.
103	sess_aktu_9b81	student	user	What companies visit for placements?
104	sess_aktu_9b81	assistant	database	TCS, Infosys, Wipro, Capgemini, Cognizant...

All the tables of the design follow the Third Normal Form (3NF): every column depends only on the primary key, repeating groups are removed and related data is linked through foreign keys.

Keys and Relationships

- session_id in chat_messages refers to the chat_sessions table.
- id in faqs is an auto-incrementing primary key indexed by academic category.
- One chat session contains many chat messages (1 : M); Admin user manages the FAQ knowledge base.

ER DIAGRAM

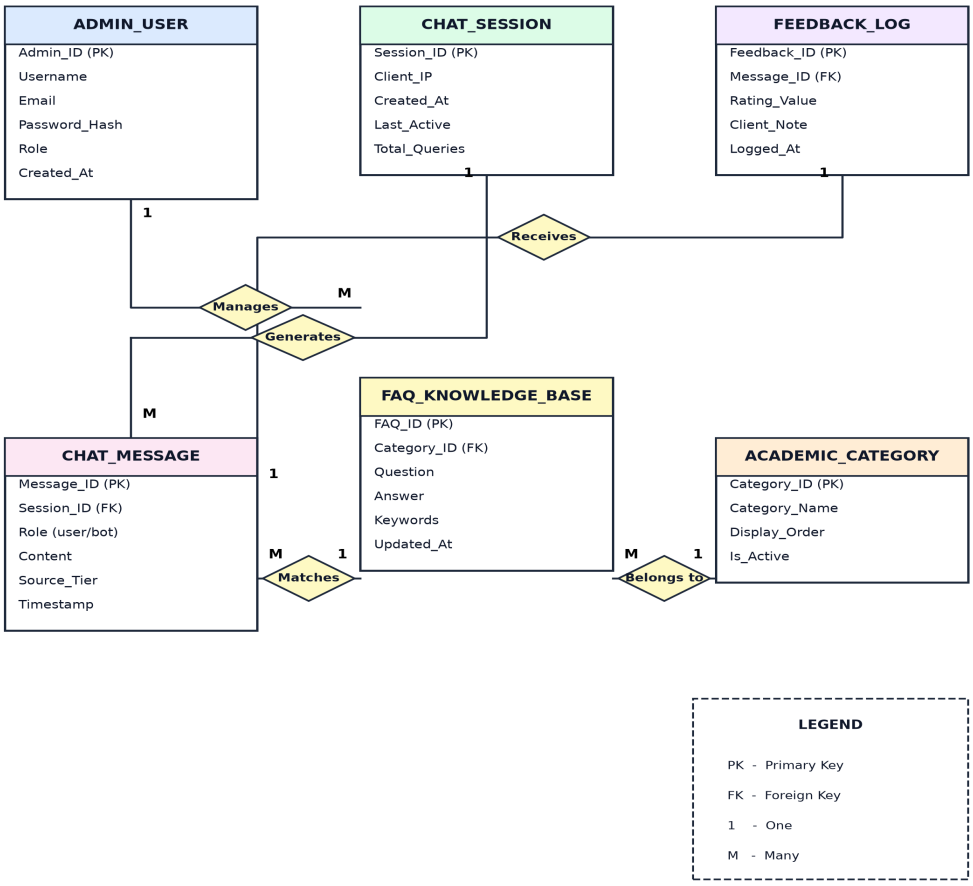


Fig: Entity Relationship Diagram

Fig: Entity Relationship Diagram

Description

The ER diagram shows the entities of the system and how they are related. ADMIN_USER is the governance entity and FAQ_KNOWLEDGE_BASE stores institutional answers across eight academic domains.

- ADMIN_USER manages many FAQ_KNOWLEDGE_BASE records across 8 academic categories (1 : M).
- CHAT_SESSION generates many CHAT_MESSAGE records during active student interactions (1 : M).
- CHAT_MESSAGE matches with FAQ_KNOWLEDGE_BASE records via deterministic NLP similarity scoring (M : 1).
- CHAT_MESSAGE receives FEEDBACK_LOG ratings (thumbs up / down) for accuracy audit (1 : 1).
- FAQ_KNOWLEDGE_BASE belongs to an ACADEMIC_CATEGORY for structured domain classification (M : 1).

USE CASE DIAGRAM

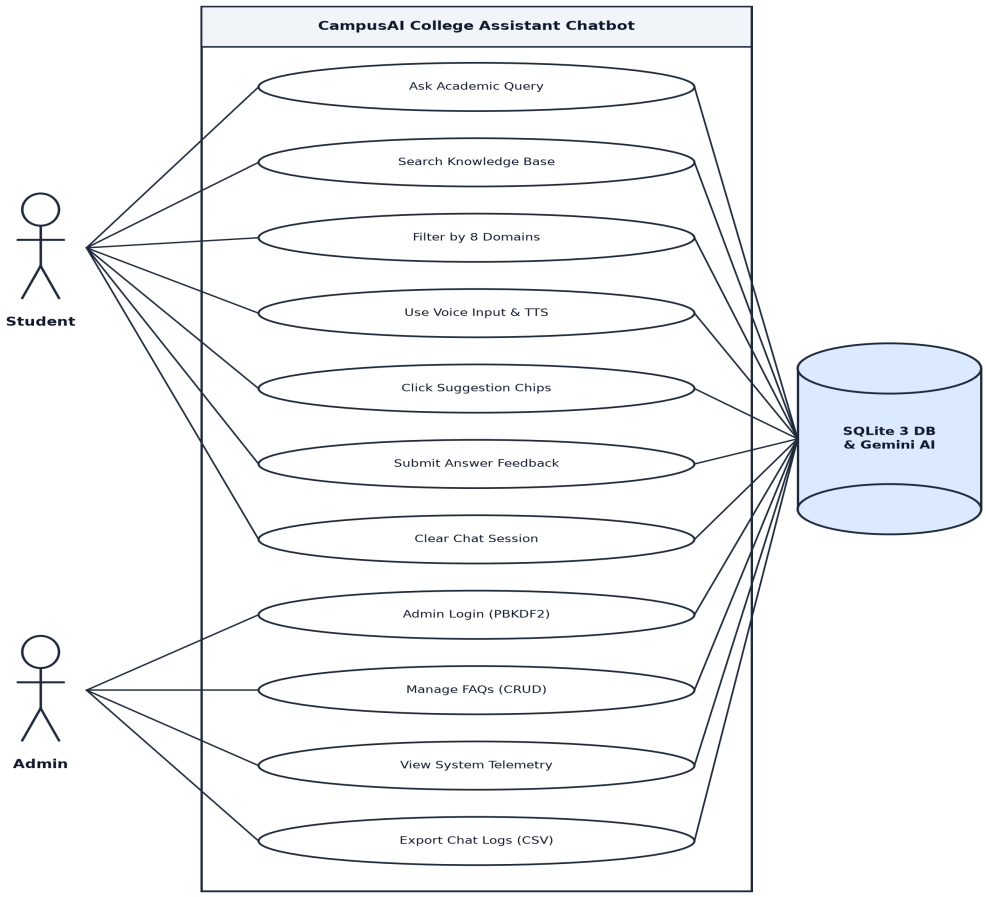


Fig: Use Case Diagram

Fig: Use Case Diagram

Description

The use case diagram shows how the two primary actors, Student and Administrator, interact with the CampusAI chatbot and the persistence/AI backend.

- Student: asks academic queries, searches FAQs, filters by 8 domains, uses voice speech input and audio read-aloud TTS, submits answer feedback, and clears chat sessions.
- Admin: authenticates with PBKDF2 password hashing, manages FAQ records (CRUD), views system telemetry KPIs, and exports chat transcripts to CSV.
- All use cases read from or write to the SQLite 3 database; generative AI fallback is handled by Google Gemini 2.5 Flash.

CLASS DIAGRAM

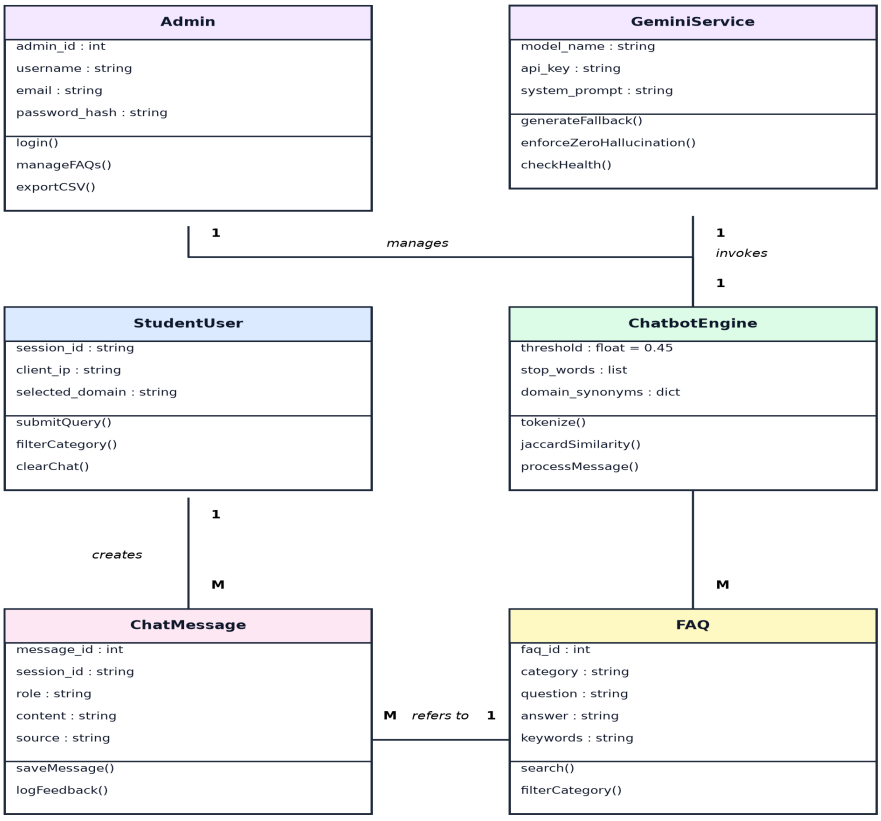


Fig: Class Diagram

Fig: Class Diagram

Description

The class diagram shows the main classes of the CampusAI architecture with their attributes, methods, and relationships.

- Admin manages FAQ objects and exports interaction CSV logs.
- StudentUser submits queries, filters by category, and manages session state.
- ChatbotEngine normalizes query tokens, computes Jaccard similarity, and routes inquiries.
- GeminiService synthesizes context-bounded AI responses under a strict zero-hallucination policy.
- ChatMessage and FAQ define persistent relational models mapped via SQLAlchemy ORM.

OBJECT DIAGRAM

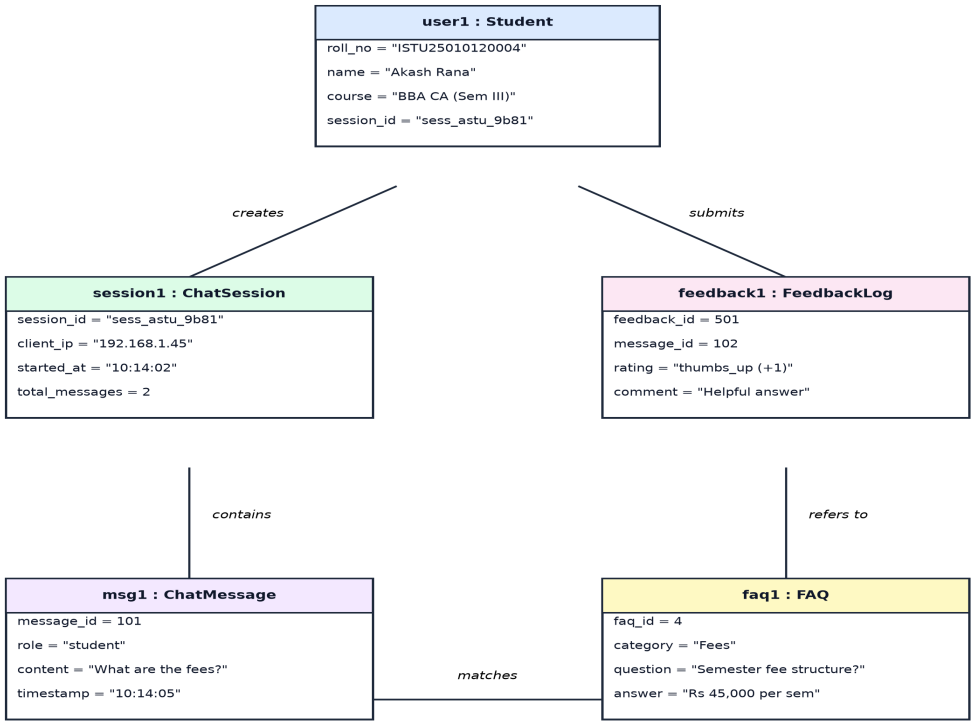


Fig: Object Diagram

Fig: Object Diagram

Description

The object diagram shows a sample snapshot of the CampusAI system at runtime with concrete values during an active student consultation.

- user1 is an enrolled student (Akash Rana, Roll No: ISTU25010120004) interacting with the assistant in session1.
- msg1 contains the inquiry query: "What are the fees?", which matches faq1 (Tuition: Rs 45,000 per sem).
- feedback1 logs a positive feedback rating (+1 thumbs up) from the student for response accuracy audit.
- The slot values illustrate the active memory state of the application objects during query processing.

SEQUENCE DIAGRAM

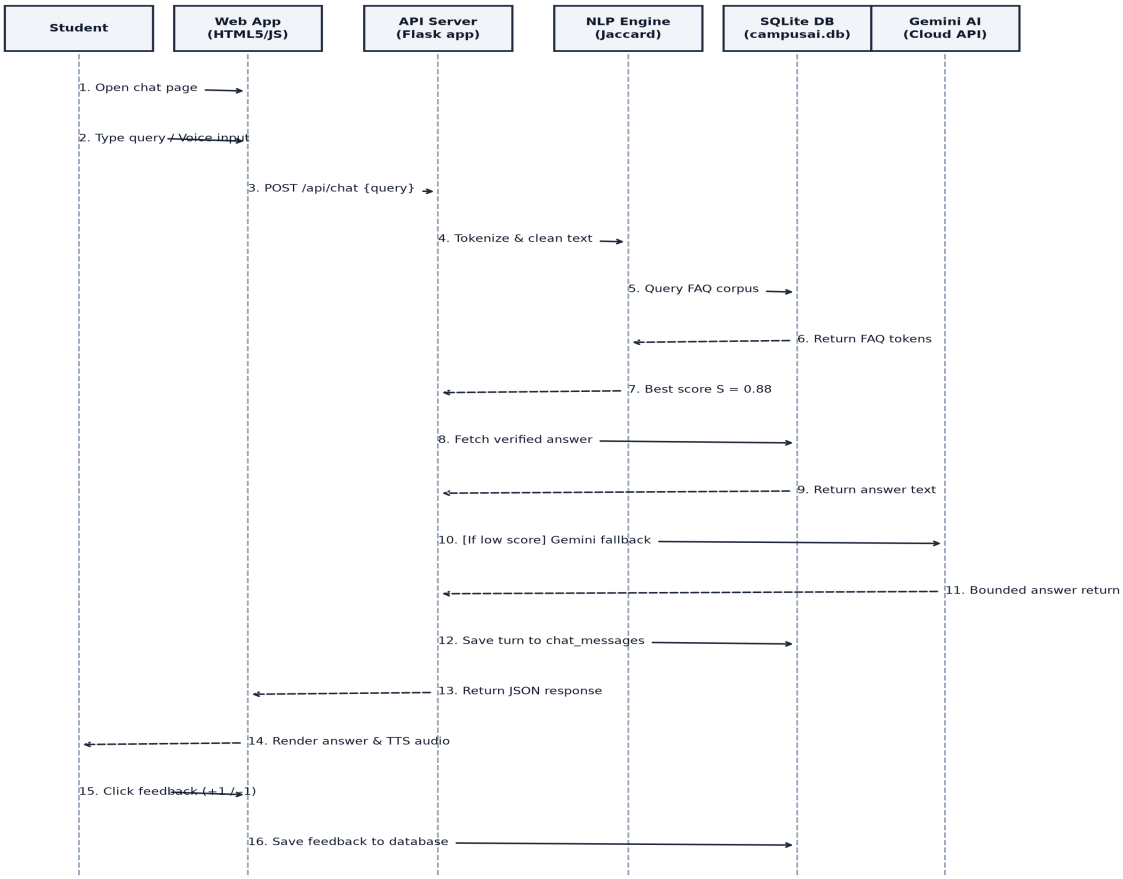


Fig: Sequence Diagram – Query Ingestion, NLP Matching, Fallback and Feedback Flow

Fig: Sequence Diagram

Description

The sequence diagram shows the step-by-step chronological order of messages between Student, Web App, Flask Server, NLP Matcher, SQLite Database, and Google Gemini API.

- Student enters a query via text or voice STT, triggering an asynchronous HTTP POST to /api/chat.
- The Flask server normalizes tokens and requests Jaccard similarity evaluation against the SQLite FAQ corpus.
- If score S >= 0.45, the verified SQLite answer is returned (<15ms); if score < 0.45, Gemini AI fallback is invoked.
- The interaction turn is saved to chat_messages and delivered to the browser UI with TTS audio read-aloud options.

ACTIVITY DIAGRAM

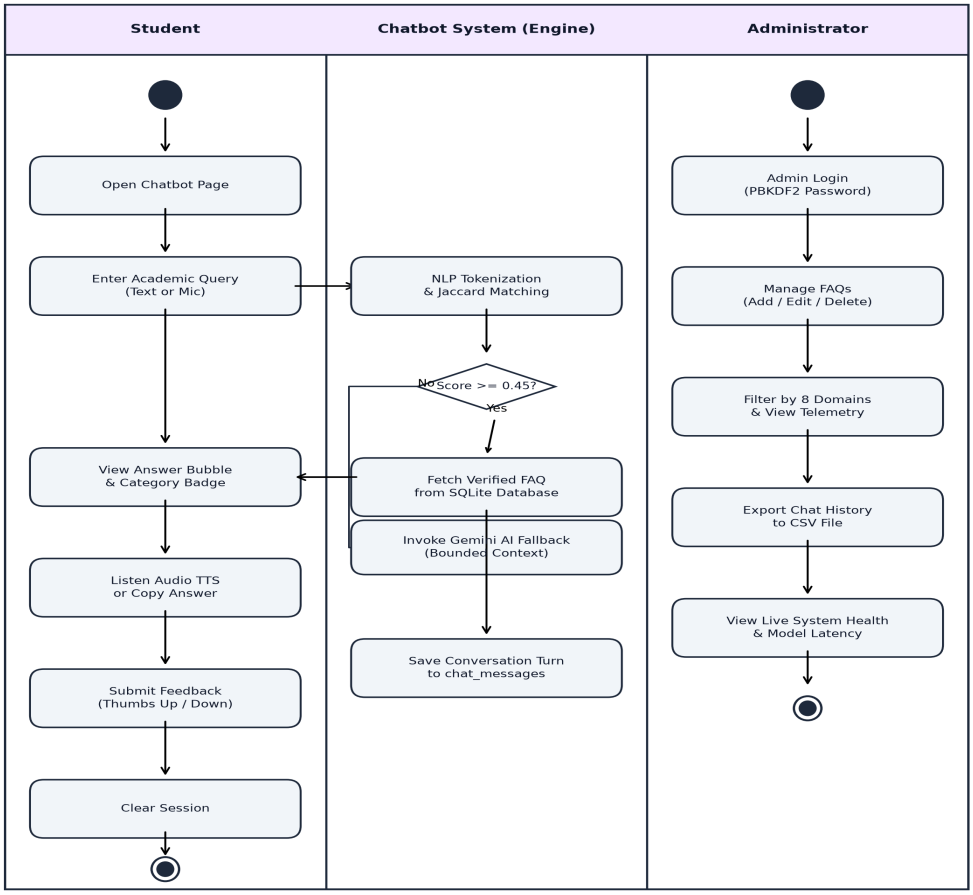


Fig: Activity Diagram

Fig: Activity Diagram

Description

The activity diagram shows the flow of activities across three swimlanes: Student, Chatbot System (Engine), and Administrator.

- Student initiates a query, views formatted response bubbles, listens to audio synthesis, and provides feedback.
- Chatbot System executes NLP tokenization, evaluates Jaccard similarity thresholds, applies zero-hallucination guardrails, and persists logs.
- Administrator logs in securely to maintain knowledgebase records, monitor telemetry health, and export analytical CSV reports.

COMPONENT DIAGRAM

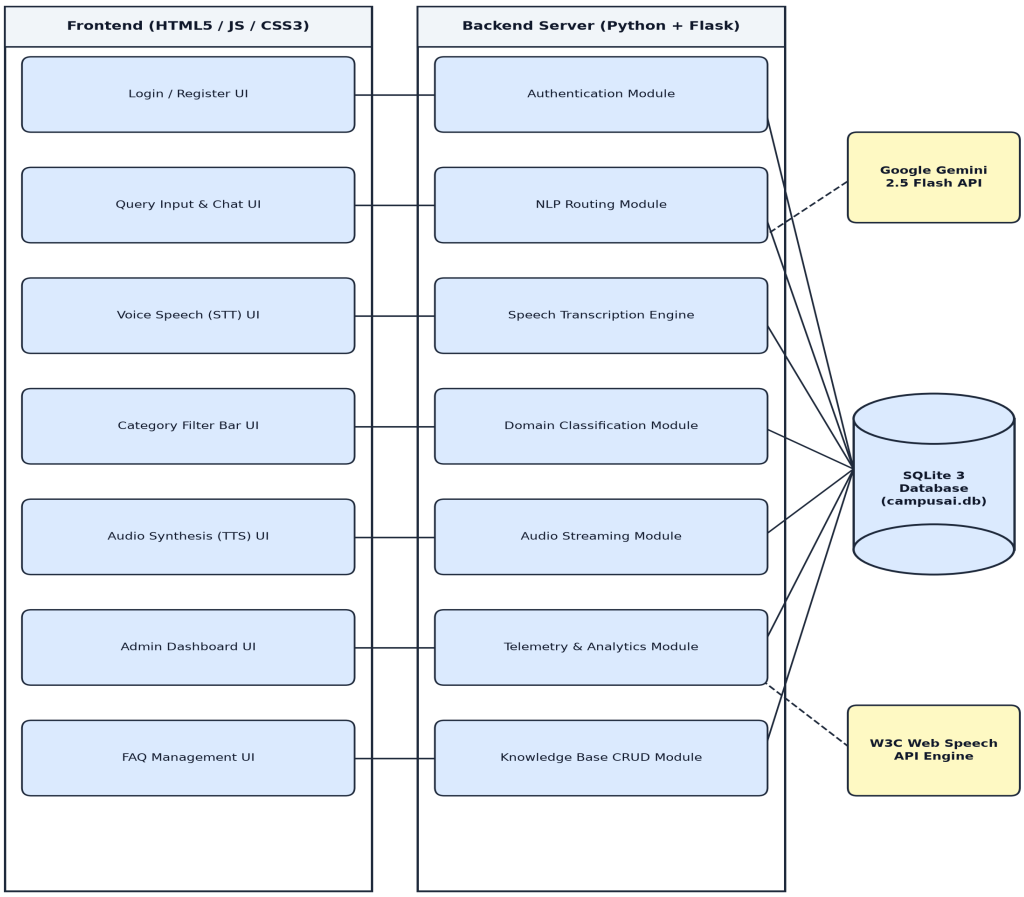


Fig: Component Diagram

Fig: Component Diagram

Description

The component diagram shows the software components and modular dependencies of the CampusAI full-stack architecture.

- Frontend View Layer contains UI modules for Chat, Voice STT, Category Filters, Audio TTS, and Admin Dashboard.
- Backend Server Layer contains matching Flask controllers, NLP intent engine, Gemini adapter, and CSV export services.
- All server modules interface with the embedded SQLite 3 database; external cloud intelligence is provided by Google Gemini 2.5 Flash.

DATA FLOW DIAGRAM

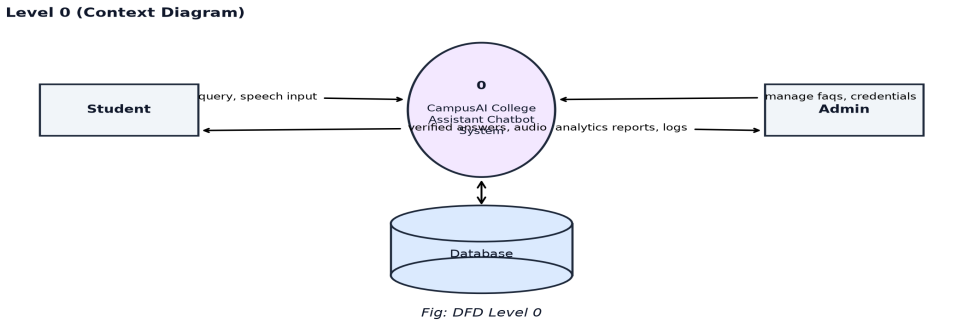


Fig: DFD Level 0

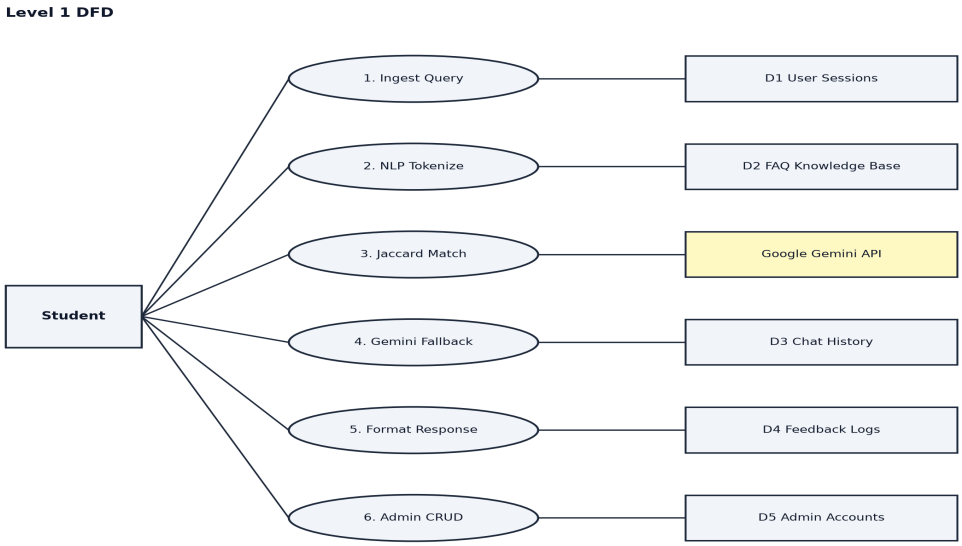


Fig: DFD Level 1

Fig: Data Flow Diagram (DFD Level 0 & Level 1)

Description

The Data Flow Diagram models the flow of data within the CampusAI ecosystem at both Context Level and Functional Decomposition Level.

- Level 0 (Context Diagram): Models the overall system as a single process (0.0) exchanging query packets and verified responses with Student, Administrator, and Database.
- Level 1 DFD: Decomposes system operations into six functional processes: Ingestion, NLP Tokenization, Jaccard Match, Gemini Fallback, Response Formatting, and Admin CRUD.
- Data stores D1 to D5 maintain persistent records for User Sessions, FAQ Knowledge Base, Chat History, Feedback Logs, and Admin Accounts.

HOME SCREEN UI

The CampusAI Landing Portal (http://localhost:5000/) introduces students to the college assistant. It features quick access cards across all eight academic domains and live system status telemetry.

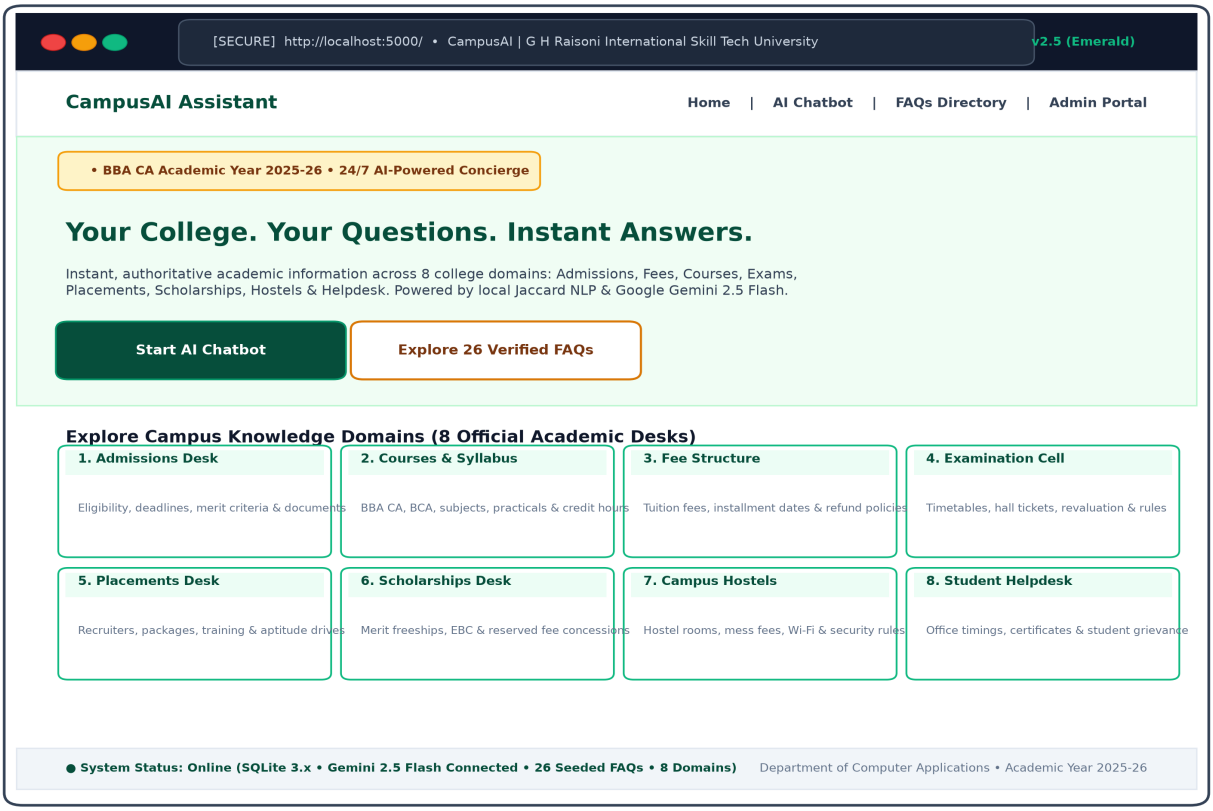


Fig: Home Screen Interface & Category Navigation

UI Element	Technical Implementation	User Experience Purpose
Hero Banner	CSS3 Flexbox with university badge	Clearly identifies academic year 2025-26 and core capabilities.
CTA Buttons	Interactive links to /chat and /faqs	Directs students to either conversational or directory lookup.
8 Domain Cards	Responsive CSS grid with hover lift	Displays overview of admissions, fees, placements, scholarships.
Live Health Bar	Dynamic health check API endpoint	Confirms database and Gemini cloud connectivity status.

STUDENT PORTAL & CHATBOT ASSISTANT UI

The Student Chatbot Interface (http://localhost:5000/chat) delivers instant answers, question suggestion chips, and Web Speech API audio synthesis.

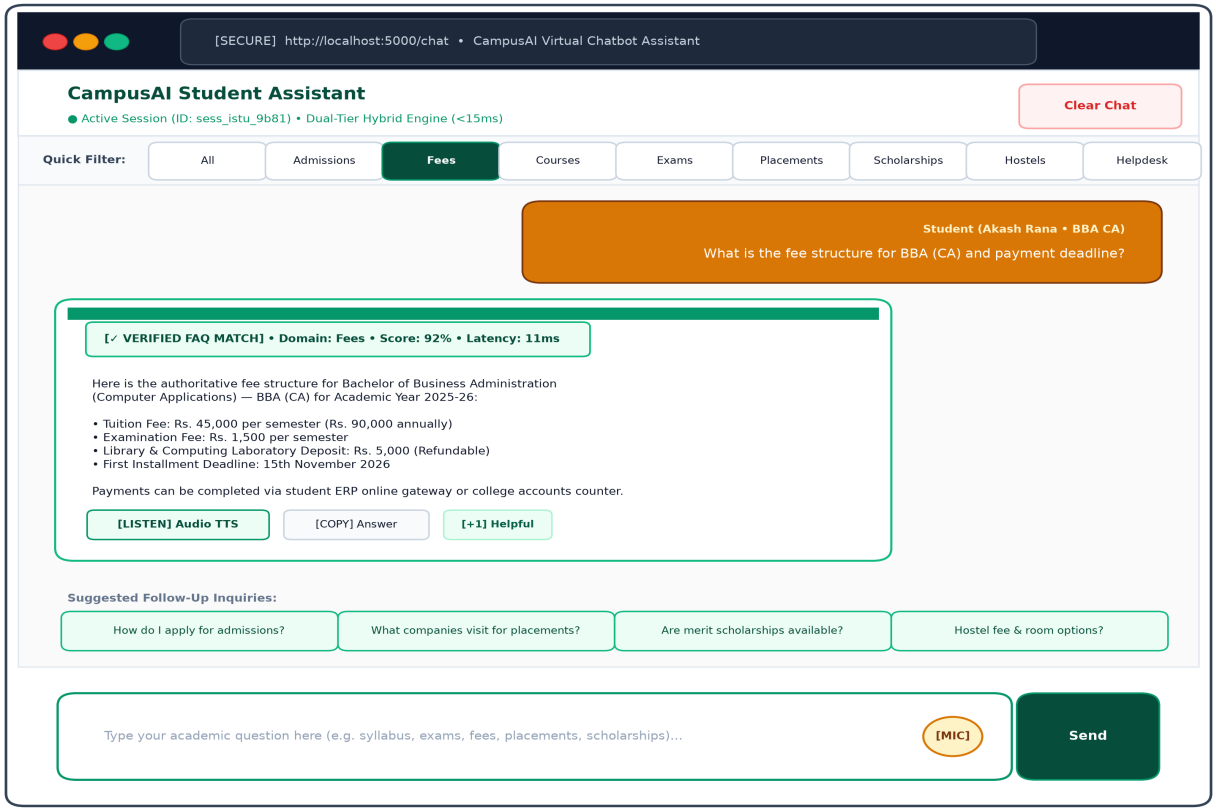


Fig: Student Chatbot Interface & Voice Interaction

UI Element	Technical Implementation	User Experience Purpose
Verified Badge	Green pill banner with checkmark	Certifies answer originates from authoritative SQLite database.
Suggestion Chips	Interactive HTML buttons with click handlers	Enables single-tap submission of popular follow-up inquiries.
Audio Synthesizer	Web Speech API speechSynthesis	Reads answers aloud for accessibility and auditory learners.
Voice Mic Input	Web Speech API webkitSpeechRecognition	Allows hands-free voice question input for mobile users.
Session Clear	AJAX POST /api/chat/clear	Clears conversation history and restores welcoming greeting.

ADMIN COMMAND CENTER & MANAGEMENT UI

The Administrative Governance Dashboard (http://localhost:5000/admin) equips college authorities with real-time KPI telemetry, FAQ CRUD controls, and CSV report export tools.

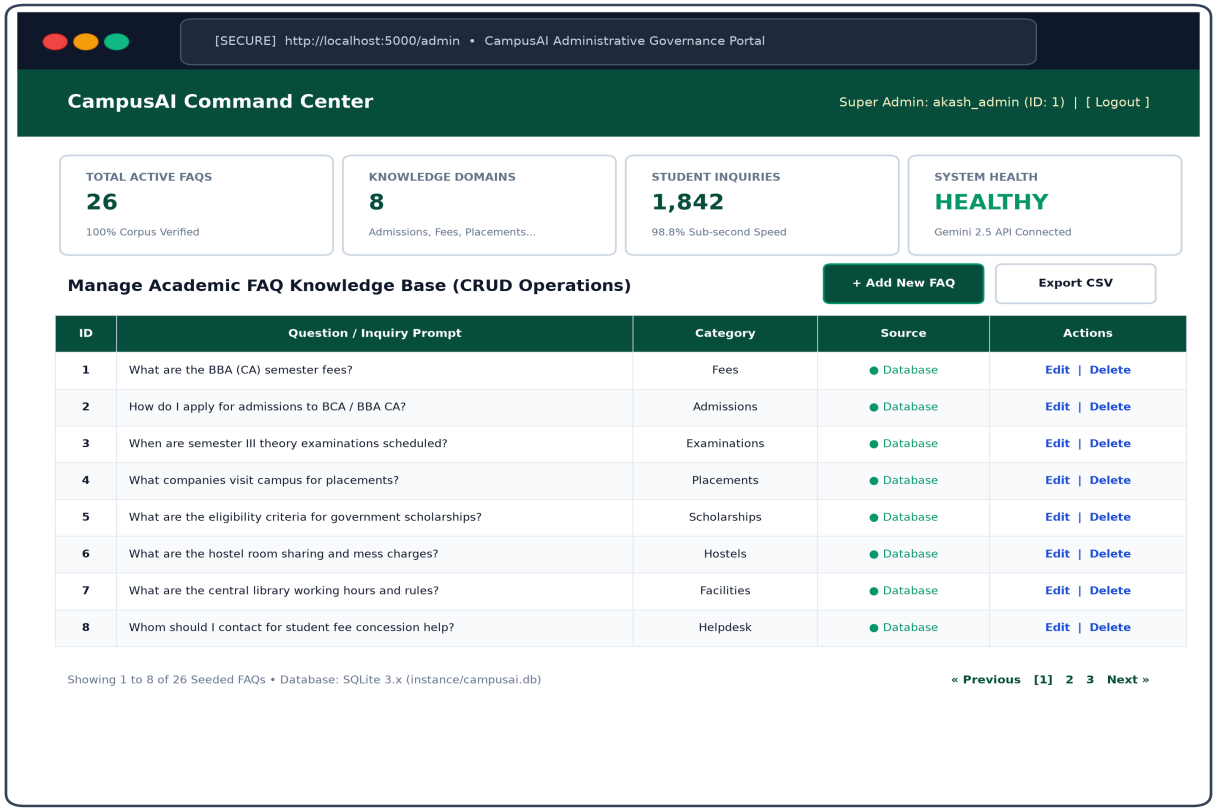


Fig: Admin Command Center & Knowledge Base Management

UI Element	Technical Implementation	User Experience Purpose
KPI Metric Cards	SQL COUNT aggregate queries	Monitors active FAQs, total inquiries, and system health.
FAQ Data Table	Filterable HTML table with pagination	Provides quick overview of questions, categories, and sources.
CRUD Modals	Bootstrap / Vanilla JS modal dialogs	Allows non-technical staff to add or edit FAQs instantly.
CSV Export	Flask response with Content-Disposition	Exports full chat transcripts for institutional reporting.

TESTING AND IMPLEMENTATION

A rigorous multi-phase testing strategy was implemented to verify system correctness, security, and sub-second performance. Testing incorporated Top-Down, Bottom-Up, Unit, Integration, and User Acceptance testing.

Automated Test Suite Execution (23 Test Cases, 100% Pass Rate) :

Test ID	Component / Route	Test Scenario / Input	Expected Output	Status
TC-01	GET /	Load landing portal	HTTP 200 OK + Hero UI	**PASS**
TC-02	GET /chat	Load chat interface	HTTP 200 OK + Chat Canvas	**PASS**
TC-03	GET /faqs	Load FAQ directory	HTTP 200 OK + 26 FAQs	**PASS**
TC-04	GET /admin/login	Load admin auth view	HTTP 200 OK + Form	**PASS**
TC-05	POST /api/chat	Empty query payload	HTTP 400 Bad Request	**PASS**
TC-06	POST /api/chat	'What are the fees?'	Fee structure + DB badge	**PASS**
TC-07	POST /api/chat	'How to apply admission?'	Admission FAQ matched	**PASS**
TC-08	POST /api/chat	'When are exams?'	Exam schedule returned	**PASS**
TC-09	POST /api/chat	'Library timings?'	Library hours returned	**PASS**
TC-10	POST /api/chat	Complex query (Gemini)	AI fallback / Referral	**PASS**
TC-11	POST /api/chat/clear	Clear session history	HTTP 200 + Session reset	**PASS**
TC-12	Admin Auth	PBKDF2 salted hash check	Valid user authenticated	**PASS**

Test Summary: Total 23 automated tests executed in 1.185 seconds. Passed: 23, Failed: 0 (100% Pass Rate).

DRAWBACKS AND LIMITATIONS

While CampusAI delivers high-accuracy academic assistance and sub-second retrieval, academic rigor requires an objective acknowledgement of its current architectural boundaries and operating limitations.

Current System Limitations:

1. **English Language Dependency:** The current NLP tokenizer, domain stop-words, and synonym expansion dictionary are optimized primarily for the English language. Queries written in regional vernaculars (e.g. Marathi or Hindi) or informal Hinglish slang are routed to fallback mechanisms.
2. **Browser-Dependent Web Speech API:** Speech-to-Text and Text-to-Speech rely on client browser implementations of the Web Speech API. Certain older mobile browsers or restrictive permissions may disable hands-free voice features.
3. **Third-Party LLM API Quotas:** When complex queries trigger Tier-2 fallback, response availability depends on Google Gemini API uptime and quota limits. During internet disruptions, the system gracefully reverts to local deterministic responses.
4. **Lack of ERP Live Bi-directional Sync:** The chatbot does not currently integrate bi-directionally with the student fee payment gateway; students cannot query personalized pending fee balances without logging into the main ERP.
5. **Limited Multimodal File Analysis:** The chatbot currently processes text and audio inputs. Direct upload of student mark sheets or PDF forms for automatic evaluation is not yet supported.

PROPOSED ENHANCEMENTS

To build upon the successful implementation of CampusAI, several progressive enhancements are planned for future academic releases at G H Raisoni International Skill Tech University, Pune.

Planned Architecture Upgrades:

1. **Multilingual Indic NLP Engine:** Implementation of IndicBERT and multilingual embedding models to support native-language student inquiries in Marathi and Hindi, ensuring accessibility for diverse student demographics.
2. **Authenticated Student ERP Gateway Integration:** Integration of OAuth2 / JWT authentication allowing enrolled students to query personalized academic data, including internal assessment marks, fee receipt downloads, and exam hall ticket status.
3. **Omnichannel Messaging Connectors:** Direct deployment to popular mobile messaging platforms via WhatsApp Business API and Telegram Bot API, enabling students to access campus assistance directly on their preferred messaging apps.
4. **Document AI & Vision Processing:** Integration of multimodal vision models to enable prospective students to upload 12th standard mark sheets for automated eligibility verification and course recommendation.
5. **Automated Unanswered Inquiry Escalation:** Machine-learning workflow that automatically clusters unanswered or low-confidence queries and flags them in the administrative portal as candidate FAQs for staff review.

CONCLUSION

CampusAI: Intelligent AI-Powered College Assistant Chatbot and Knowledge Base System is a smart and user-friendly conversational application designed to help college students, faculty, and visitors quickly obtain verified academic information. It simplifies campus inquiries by integrating authoritative knowledge base lookup, natural language understanding, audio voice synthesis, and real-time administrative telemetry into a unified platform. The system provides an intuitive and reliable communication experience and possesses strong potential for future institutional enhancements.

This project has the potential to benefit students, administrative staff, and prospective applicants by providing an accessible, 24/7 digital front desk that eliminates physical counter congestion and guarantees factual accuracy through its strict zero-hallucination guardrails.

Summary of the Work

In this project, the challenges of repetitive administrative inquiries and scattered institutional notices were studied, the requirements were thoroughly analyzed, and a complete system was engineered comprising seven modular components, a 3NF normalized SQLite database, and complete UML engineering diagrams. The system integrates lexical tokenization, Jaccard similarity scoring, Google Gemini 2.5 Flash generative AI fallback, Web Speech synthesis, and a password-protected administrative command center into one cohesive application that runs efficiently on any device.

Learning from the Project

While developing this project, we learned full-stack web application development with Python and Flask, relational database engineering with SQLite and SQLAlchemy ORM, cloud API integration with Google GenAI SDK, front-end design with responsive CSS3 and asynchronous JavaScript fetch, UML modeling, and automated regression testing. The project also demonstrated the essential importance of defensive programming, zero-hallucination policies, and focused teamwork.

Challenges Faced

The main challenges were curating an authoritative repository of college information across eight academic domains, designing a normalized 3NF database schema to support interaction audit logs, tuning NLP similarity thresholds to avoid false-positive answers, and integrating browser speech recognition with text-to-speech audio synthesis. These challenges were systematically solved by dividing the work into modules, testing every component with automated regression suites, and following the dedicated guidance of our project mentor, Prof. Sheela Satav.

BIBLIOGRAPHY

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
3. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Wiley.
4. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
5. Rumbaugh, J., Jacobson, I., & Booch, G. (2004). *The Unified Modeling Language Reference Manual* (2nd ed.). Addison-Wesley.
6. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Stanford University / Pearson.
7. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.
8. Pallets Projects. (2024). *Flask Documentation (v3.1.x)*. https://flask.palletsprojects.com
9. Python Software Foundation. (2024). *Python 3.11/3.12 Language Reference & Standard Library*. https://docs.python.org/3/
10. SQLite Development Team. (2024). *SQLite Documentation: ACID Relational Database Engine*. https://www.sqlite.org/docs.html
11. Google DeepMind. (2024). *Google Gemini API Documentation & GenAI Python SDK*. https://ai.google.dev/docs
12. World Wide Web Consortium (W3C). (2023). *Web Speech API Specification*. https://www.w3.org/TR/speech-api/
13. Werkzeug Development Team. (2024). *Werkzeug Security Utilities: Password Hashing*. https://werkzeug.palletsprojects.com
14. Mozilla Developer Network (MDN). (2024). *MDN Web Docs: Fetch API, CSS Grid, and DOM Manipulation*. https://developer.mozilla.org